Residual MLPs on MNIST, CIFAR-10, and ImageNet32

Taylor Turner

AI 4200 Project 1 | 5000-level section | October 30, 2026

I. Final models

The biggest gain came from changing how the model sees an image, as predicted by Dr. Roberts in lecture. A flattened residual MLP was enough for MNIST. For CIFAR-10 and ImageNet32, I used many small shared MLPs to build an image representation before the residual classifier. The overwhelming majority of experimentation was done on the CIFAR dataset as it was a reasonably sized dataset that closely approximated the gains I would find on ImageNet and allowed for much faster iterations. Ultimately very few runs were done on ImageNet, even as performance continued to grow up across each.

These final scores come from the fresh seed-0 instructor harness at commit b5ab6e1, with CUDA on an A100-SXM4-80GB. Total training took 40.33 minutes. The five grading-source files at main commit 7355660 match this successful qualification byte-for-byte; later changes concern documentation and packaging. MNIST and CIFAR baselines are historical journal measurements on Apple Metal. The ImageNet baseline was recovered on October 28 with the unchanged instructor source, seed 0, and an A100; it took 15.42 minutes. These comparisons measure complete recipes across different run conditions.

All tasks use cross-entropy, batch 256, a constant learning rate, and a seeded 10% validation split from the supplied training data. Inputs map from uint8 to [-1, 1]. AdamW uses betas 0.9/0.999 and epsilon 1e-8. Validation accuracy selects the best complete epoch, with earlier ties preferred. No ensemble, pretrained weights, distillation, extra training data, or checkpoint loading is used. Each train() call starts fresh. The instructor harness computes the test scores [1].

II. What mattered most

Three comparisons explain most of the useful progress. These are measured recipe comparisons, not isolated causal effects. The run IDs and full settings remain in LOG.md.

AdamW was useful, but it was not a magic bullet. The flattened CIFAR models still reached a plateau. The first window model was worse than the better flat models. Widening it helped, then the coverage check exposed a more basic problem: the unpadded hierarchy only used the top-left 17x17 region. Padding kept the bottom and right edges and added a sixth reduction stage.

The work moved through three broad eras: understanding the flat baseline, testing optimization and regularization, and changing the image representation with regularization. The core idea that I was exploring, especially in the last stage, was the model was more than capable of extracting signal as long as I surfaced it. And regularization is the means by which I surfaced signal. MNIST largely stayed in the first two. The window hierarchy defined the last major CIFAR and ImageNet change. The later fresh harness check separated reproducible submission scores from exploratory results.

A. Experiment history: CIFAR-10 and ImageNet32

Figures 1-3 show the saved-history snapshot through the October 27 qualification. The x-axis orders experiments; it is not elapsed time or training epochs. Downward triangles retain results below each clipped y-axis. The figures do not show the later baseline recovery or extra-block experiment. Historical continuation is labeled separately from fresh training. The full plot data retain exact values and run IDs [2].

B. Training curves: fitting the data was not enough

Figure 4 compares cifar10-20261023-112130-b25190a4 with cifar10-32ae8cde. The earlier curve is a saved development model, not the instructor's original baseline. Recipe, seed, and hardware differ. The comparison shows the change in overall training behavior; it does not isolate one architectural component. Training accuracy uses minibatch predictions, with augmentation when enabled [2].

The earlier CIFAR model could fit the training set much better than its held-out images. This was the reason to try regularization and augmentation, but stronger regularization did not remove the representation limit. The window model reached a much higher validation score. Its validation loss still stopped improving before training ended, so selecting a validation checkpoint remained useful. I specifically chose to train more steps than was necessary and then select the best validation, knowing that it would approximate the test accuracy without introducing accidental data leakage. By default, the project didn't necessarily call out using a three split, but I realized that by targeting test directly, even though it wasn't in the training set, I was introducing bias, which is a big reason why I ultimately had to remove it from my decisions.

The ImageNet result is especially useful. Training loss kept falling, but validation loss reached its minimum at epoch 15. Best validation accuracy came later, at epoch 23. Accuracy and cross-entropy answer different questions: a model can classify more examples correctly while becoming more confidently wrong on others. I selected by validation accuracy and kept the loss curve as evidence of the remaining generalization problem.

Task	Original baseline test	Final test	Gain, points	Train, min
MNIST	95.65%	99.15%	+3.50	14.70
CIFAR-10	48.21%	74.76%	+26.55	3.82
ImageNet32	6.766%	19.960%	+13.194	21.81

Setting	MNIST	CIFAR-10	ImageNet32
Image input	Flat 784 pixels	Six shared 2x2 window MLP stages	Same six window stages
Window hidden / output	Not used	128 / 64	128 / 64
Residual depth; width; hidden	7; 256; 512	2; 512; 1024	2; 512; 1024
Block / activation	Pre-LayerNorm MLP / GELU	Normalized multiplicative gate / GELU	Same gated block / GELU
Initialization	Xavier normal, zero biases	Xavier normal, zero biases	Xavier normal, zero biases
Optimizer; learning rate	SGD; 0.05; no momentum	AdamW; 0.001; AMSGrad	AdamW; 0.001; AMSGrad
Weight decay / dropout	0.0001 / none	0.00001 / 0.1 on branch	0.00001 / 0.1 on branch
Augmentation	Affine, brightness, noise, erasing	Affine, brightness, noise, erasing, horizontal flip	Off
Epoch / time ceiling	500 / 15 min	60 / 15 min	25 / 60 min
Selected / completed	270 / 382	56 / 60	23 / 25

Change in LOG.md	Validation before / after	Gain, points	What the evidence supports
77 to 79: SGD to AdamW recipe	52.48% / 54.36%	+1.88	Adaptive optimization helped this flattened CIFAR recipe. The learning rate also changed.
137 to 138: wider windows and classifier	49.98% / 60.80%	+10.82	The narrow hierarchy lacked useful capacity. Several widths changed together.
138 to 139: padding and full coverage	60.80% / 76.10%	+15.30	Retaining the image edges helped this hierarchy. Coverage and stage count changed together.

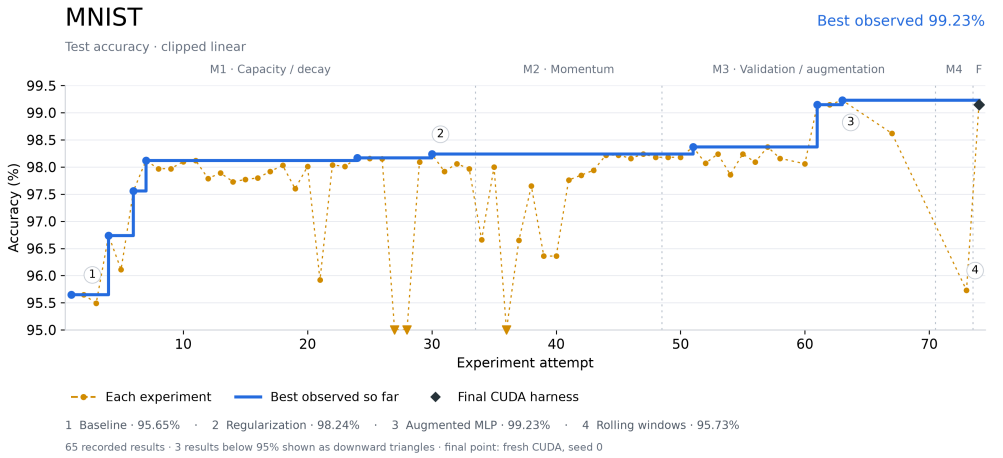


Fig. 1. MNIST experiment history.

Qualified run	Train loss, first to last	Validation loss, first to last	Minimum validation loss	Selected accuracy epoch
MNIST	0.742 to 0.012	0.218 to 0.027	0.024 at 361	270
CIFAR-10	3.574 to 0.457	1.909 to 0.796	0.736 at 56	56
ImageNet32	5.651 to 3.252	5.013 to 4.111	4.070 at 15	23

The earlier 20.004% ImageNet test result used continued weights. It is not evidence that the grading function can reach that score from scratch. The fresh 19.960% result is the qualified score. I kept the continuation in the history rather

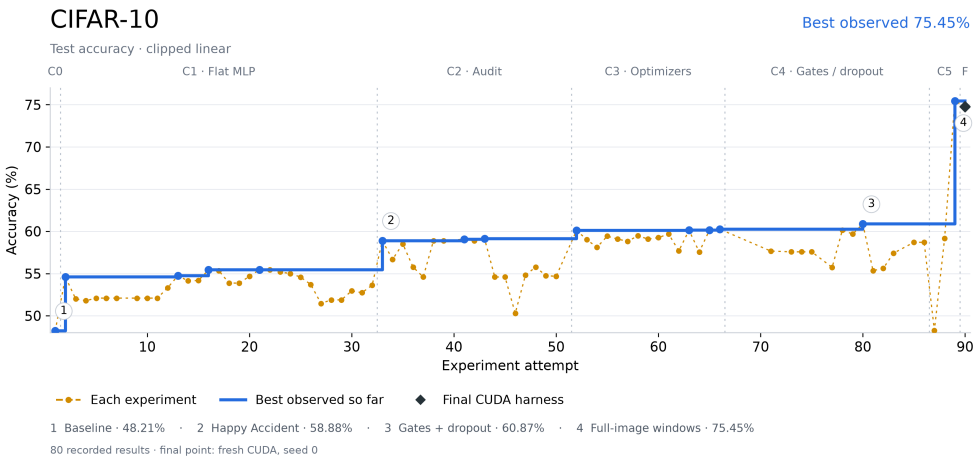


Fig. 2. CIFAR-10 experiment history.

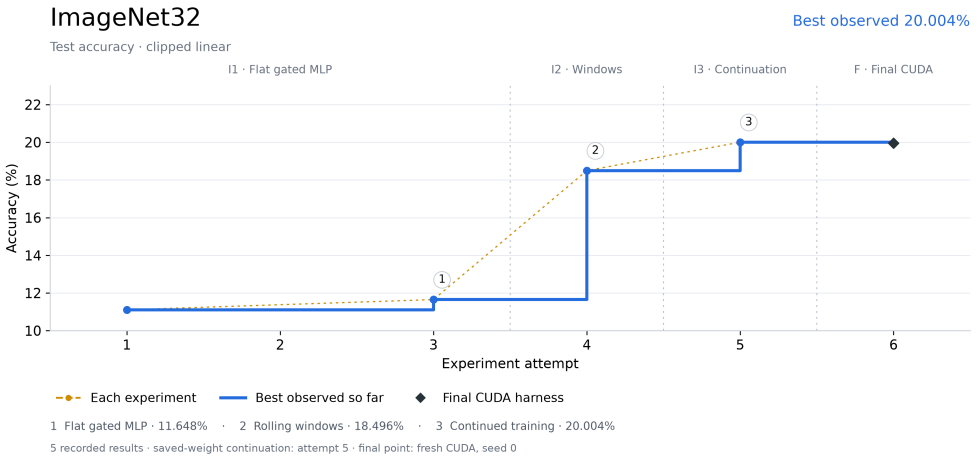


Fig. 3. ImageNet32 experiment history.

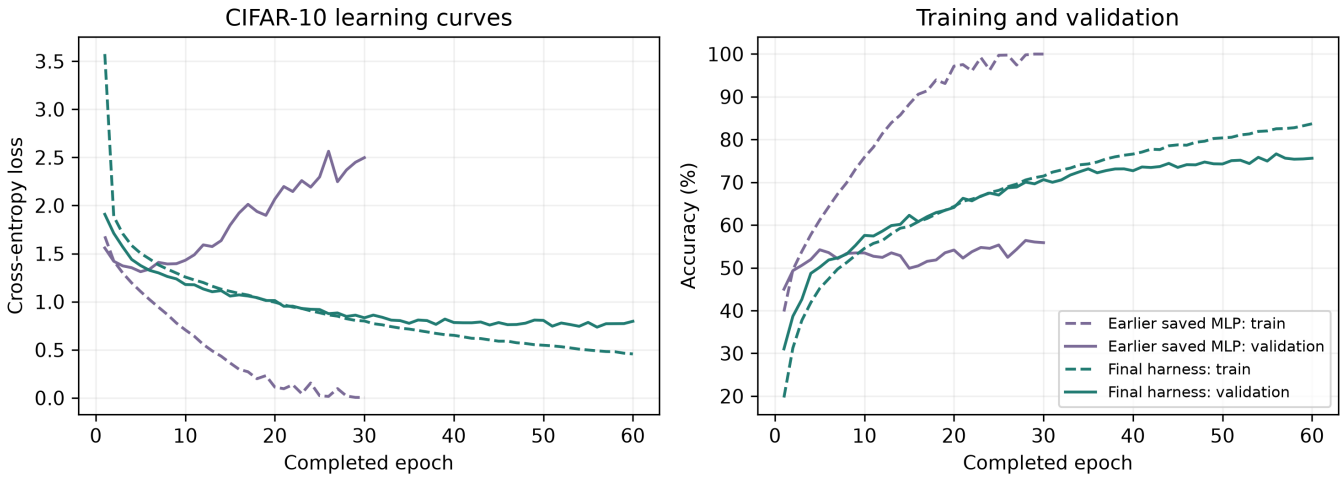


Fig. 4. Earlier flat CIFAR experiment versus the qualified final window model.

than treating it as the final submission.

III. ImageNet Training Dynamics

I measured the same 32 training examples in eval mode at initialization, epochs 1, 5, 10, 15, 20, and 25, and the selected

epoch-23 checkpoint. The probe records 20 linear weight matrices, 22 activation outputs including both residual streams, and both branch-to-stream ratios. Activation variance is measured over each recorded output tensor; gradient RMS summarizes each weight-gradient matrix. A branch ratio divides RMS(block output - block input) by RMS(block input), and an update ratio divides RMS(weight change) by RMS(weight). A copied model and actual optimizer state receive one probe step. Live training is unchanged [3].

Activation variance. The stream is not uniform through the network. At initialization, one late window linear output has variance about 2.07e-8. The first classifier branch then dominates its tiny input. By the selected checkpoint, residual-stream variances after blocks 1 and 2 are 0.0902 and 0.1367. Internal value and gate projections reach much larger variance, up to 990.6, before their normalization. That large internal value is not the residual-stream variance. The normalizations help separate internal scale from the added branch scale.

I also wanted to call attention to the zero epoch variances being a massive outlier. That section's entire purpose was to extract more fine-grained detail by asking the model to look at a smaller section at a time. At initialization, activation variance falls through the feed-forward window MLPs and rises in the residual section. After the first epoch, this pattern is less extreme. These readings describe activation scale; they do not by themselves show vanishing or exploding gradients.

Branch-to-stream ratio. Block 1 starts near 7915 because the incoming representation is tiny, not because the whole network has already diverged. By epoch 5 its ratio is 1.075. At the selected checkpoint, ratios are 0.9455 and 0.6994. Neither residual branch is close to unused.

Gradients and updates. Selected-checkpoint RMS gradients range from 0.000636 to 0.06757. The early window weights receive larger gradients than the classifier weights, but the sampled layers show no zero-gradient bottleneck. Update-to-weight ratios range from 0.00118 to 0.00451, compared with 0.00857 to 0.02775 at initialization. Most late-training probe steps are within a few times the assignment's 1e-3 reference. All recorded values are finite; one batch cannot prove stability on every minibatch. Gradient/update depths 1-12 are window linears, 13 is the input projection, 14-19 are classifier-block linears, and 20 is the output head.

The update ratios describe a fixed-batch step with copied optimizer state, not the live random minibatch step. GELU has no tanh saturation or ReLU dead-unit criterion as defined by this assignment, so that specific saturation measurement does not apply. Plot positions follow forward order, with the exact module map in layer-index.json [3].

A. 5000-level ablation: which blocks matter?

I bypassed one residual classifier branch at a time, retaining its identity skip and every other block. I did not retrain. Each evaluation used all 128,116 validation examples from the training split. The full selected model scored 22.0222% [3].

Block 1 matters more by both measurements. Block 2 has the smaller ratio and smaller accuracy drop, but losing 3.61 points is still substantial. Neither is an obvious free removal. With only two branches, this is agreement in ordering, not evidence of a general predictive relationship. Bypass results describe dependence of the trained model; they do not tell me how a separately trained shallower model would perform.

The window stages do not have identity skips. I therefore changed their inputs rather than deleting whole stages. At one stage, I repeated either the top-left value or the 2x2 mean in all four positions. The learned MLP, strides, padding, and output shape stayed intact. Mean-only accuracy fell to 0.5823% at stage 1, 16.4515% at stage 4, and 6.9367% at stage 6. Spatial arrangement matters throughout the hierarchy, particularly near its ends. These interventions also shift feature distributions; they are not a smaller-model benchmark. All 12 controls and restoration checks are preserved [4].

IV. What did not work

More capacity before compression. The spatial ablations made me wonder whether another residual block could extract more signal before the last reduction. I tested one fresh seed-0 candidate. It added a 256-to-512-to-256 residual MLP before the final 2x2-to-1x1 stage. The split, shared initial parameters, optimizer, and 25-epoch recipe matched the qualified model. No retry, continuation, or test-set evaluation followed [5].

The extra block lost 0.9156 points and fit the training data worse. I cannot explain that as overfitting alone. Its best accuracy and minimum validation loss occurred at the cutoff, so slower optimization is plausible. The trial only says that this added block did not help under this recipe and budget. I kept the original architecture.

Making the flat model larger. In the MNIST journal, width 256 reached 99.99% training accuracy but only 98.04% test accuracy. That was a small held-out gain despite nearly perfect fit. CIFAR showed the same broad problem more strongly: fitting flattened pixels did not produce enough generalization. Capacity and regularization had to be considered together.

The first window hierarchy. Change 137 reached only 49.98% validation accuracy. The idea alone was not enough: its narrow features and incomplete image coverage were weaknesses. Changes 138 and 139 addressed both, but they did not isolate the individual effects.

Incomplete or discarded designs. The history includes crashes, unfinished block edits, non-residual trials, and MaxPool2d experiments. Those trials remain in the log as failures or discarded directions. The frozen grading route uses the allowed linear, activation, normalization, and residual building blocks, with no pooling module. The separate Google MLP-Mixer exploration has no full-dataset result and was not selected.

V. Across the three tasks

MNIST did not need the image hierarchy. Its digits have a simple shared structure and only ten classes. The seven-block flat model could reach 99% without learning a large collection of local color and texture features. Keeping that earlier recipe was better supported than applying the latest architecture to every task.

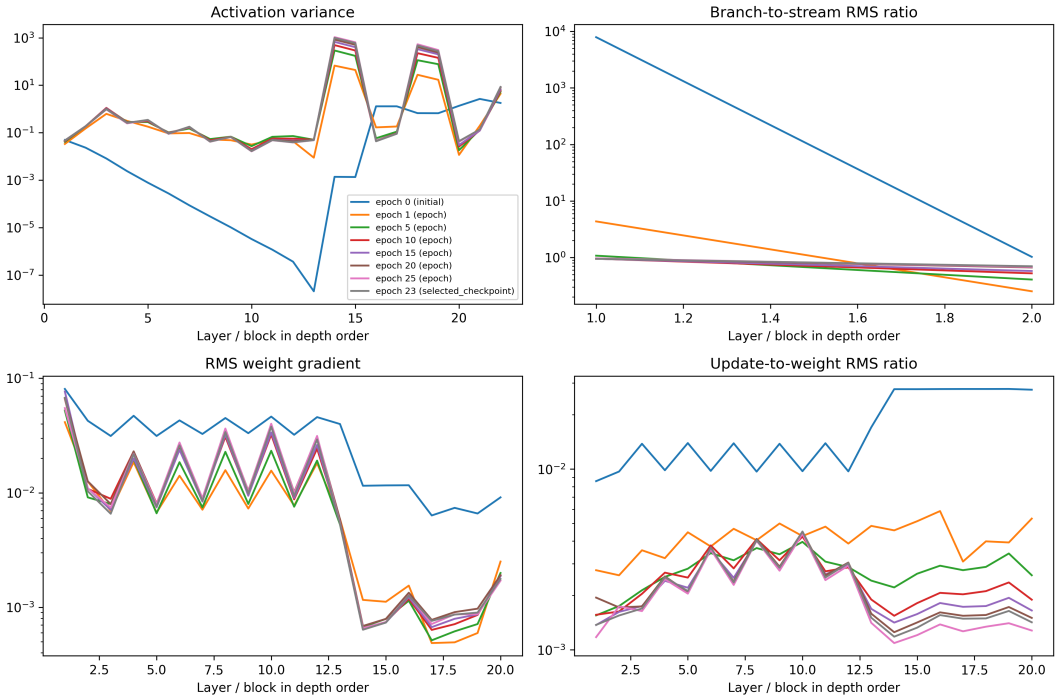


Fig. 5. Fixed-batch ImageNet measurements across training. All four y-axes are logarithmic.

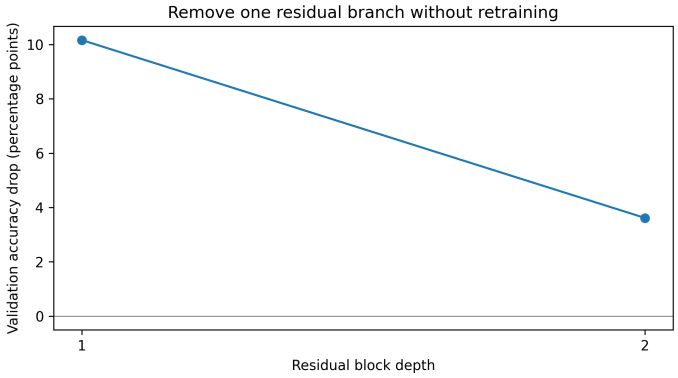


Fig. 6. Accuracy drop when each residual branch is bypassed.

Branch removed	Validation accuracy	Drop, points	Original branch / stream RMS
Block 1	11.8588%	10.1634	0.9455
Block 2	18.4091%	3.6131	0.6994

Measure	Qualified model	Extra residual block
Best validation accuracy	22.0222%	21.1067%
Selected / completed epochs	23 / 25	25 / 25
Minimum validation loss	4.069596	4.101746
Final training loss	3.252028	3.419130
Training time, min	21.81	23.52

CIFAR-10 has the same number of classes, but much greater variation in object position, color, texture, and background. Flattening gives each pixel its own position in a large vector. It asks the network to learn spatial relationships from scratch. Shared window MLPs reuse the same local operation across positions, then combine those features over larger regions. This changes the model's assumptions about images. It was a more useful change than continuing to scale the flat stream.

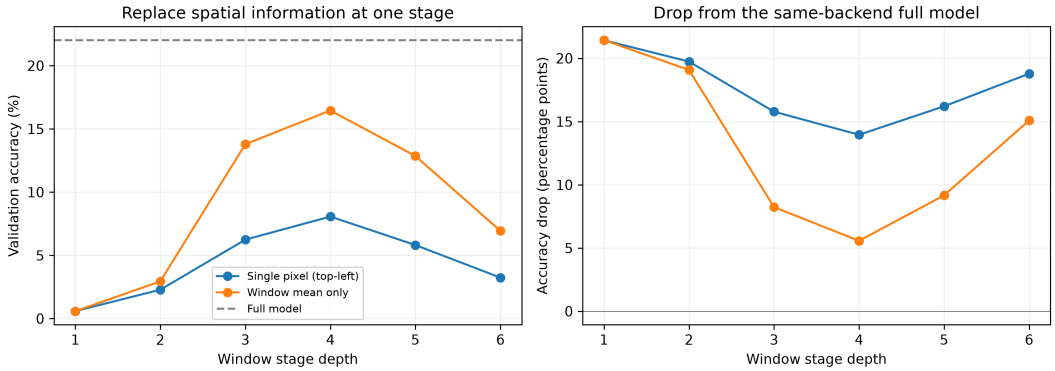


Fig. 7. Additional spatial-information controls at all six window stages.

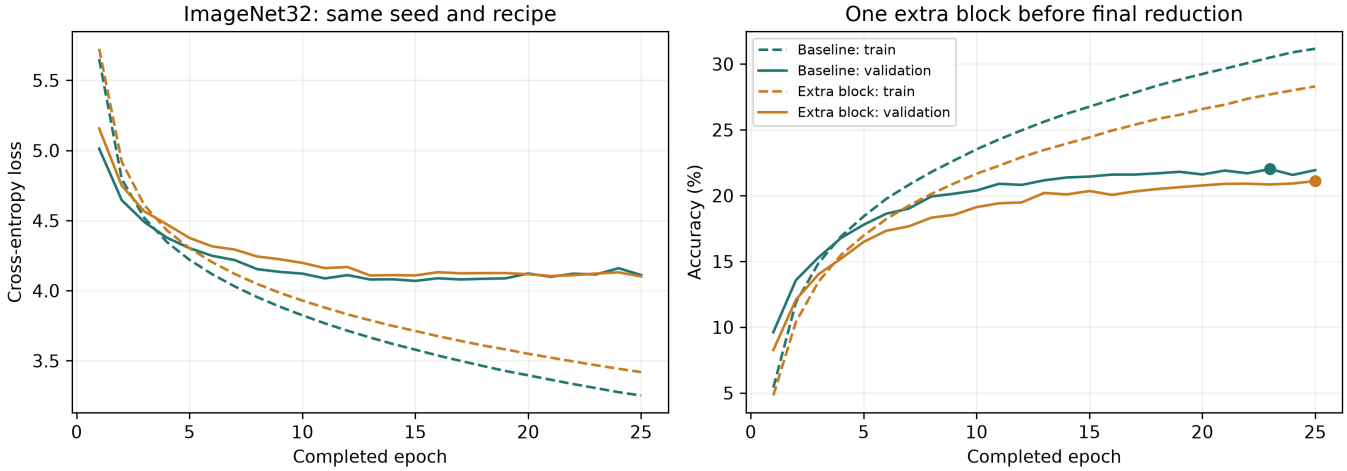


Fig. 8. One extra-block candidate versus the qualified ImageNet model.

ImageNet32 adds 1000 classes and many more training images, but each image still contains only 32x32 pixels. Fine distinctions may be missing at that resolution. The window representation transferred, but its absolute accuracy stayed far below CIFAR. ImageNet augmentation was off in the selected recipe; I do not have a controlled result proving that turning it on would improve this final model.

The comparison does not establish a hard accuracy limit for MLPs. It shows a limit of the recipes I tried. The local hierarchy helped, but compression, optimization, and generalization still constrained ImageNet. The ablations show that the trained model uses both classifier branches and the spatial hierarchy. They do not imply that adding more layers must help.

VI. Techniques Beyond the Basic Baseline

Shared window MLPs and padding. At each stage, Tensor.unfold gathers a 2x2 neighborhood. A shared Linear(4C,128), GELU, Linear(128,64), GELU maps it to 64 features. The first stage uses stride 1; later stages use stride 2. Bottom/right zero padding preserves odd-sized edges. Spatial sizes are 32, 31, 16, 8, 4, 2, and 1. The final 64 features project to width 512 before the residual classifier. This uses linear layers and reshaping, while introducing locality and weight sharing. That inductive bias resembles convolution mathematically; the implementation does not use a convolution layer. My journal's early claim that the operation was not shared was incorrect. MLP-Mixer was related reading, not the final architecture [6].

Multiplicative residual gating. For stream x, compute z = LayerNorm(x), value = LayerNorm(Linear(z)), and gate = LayerNorm(GELU(Linear(z))). The block returns x + Dropout(Linear(value * gate)). One learned path modulates another through elementwise multiplication. This is not a sigmoid gate or attention. I kept it as part of the documented recipe, but the comparisons do not establish that gating alone caused the gains. The journal linked Activator during this exploration; it did not reproduce that vision transformer [7].

GELU and pre-normalization. GELU scales an input by a smooth Gaussian-based gate rather than applying ReLU's hard cutoff [8]. LayerNorm normalizes each example across its feature dimensions. Applying it before the branch controls the branch's input scale while leaving the skip available. This helps explain how large internal projections can coexist with moderate residual-stream variance.

A. Technique notes and reproducibility

AdamW and AMSGrad. Adam uses running first and second gradient moments to scale updates. AdamW applies weight decay separately from that gradient scaling [9]. AMSGrad retains a running maximum of the second-moment estimate in the update denominator [10]. I used the documented combined recipe. The log supports an improvement over one SGD recipe, but does not isolate AMSGrad's contribution.

Other explored designs. Muon approximately orthogonalizes momentum-based updates for selected two-dimensional hidden weights. A separate AdamW optimizer handles the remaining parameters in this implementation. The trials targeted the flat-model plateau, but the final recipes use SGD or AdamW. The separate Mixer design builds linear patch embeddings, then alternates residual MLPs across patch positions and channels before averaging tokens [6]. It has no completed full-dataset score here, so it supports no accuracy claim.

Dropout and augmentation. Dropout randomly masks branch outputs during training and is disabled in eval mode. Augmentation creates transformed training views without adding external images. MNIST uses rotation (probability 0.5, magnitude 10 degrees), translation (0.5, 2 pixels), scaling (0.5, 0.1), shear (0.25, 5 degrees), brightness (0.25, 0.1), noise (0.25, 0.05), and erasing (0.1, 0.05). CIFAR probabilities are 0.45/0.5/0.55/0.5/0.45/0.5/0.5 in that order; translation magnitude is 4 pixels, plus horizontal flip at 0.5. Exact sampling and normalization are in submission.py and training_core.py. No test-time augmentation is used.

Evidence limits. Some early choices used test scores before I switched to validation-based selection. I cannot rewrite those choices as validation-only experiments. LOG.md labels later reconstructions and estimated expectations; they are not predictions written before the run. Missing historical curves and settings remain unknown. The final choices have saved run IDs, source settings, and a separate fresh harness check. Most comparisons use one seed and sometimes change several settings, so I treat the measured gains as recipe evidence rather than proof about a single mechanism.

The original baseline has 30 tanh residual blocks, width 128, hidden 256, Xavier initialization, SGD at 0.05, batch 256, and ten epochs. The ImageNet recovery used instructor commit b56785a206069671420fbf5c6bbd6efb99f467b9. The final public API is train(task, X_train, y_train, seed=0, device="cuda") and predict(model, X_test). Analysis scripts run separately; diagnostics are off by default. The repository retains the log, code, executable harness, exact settings, and compact evidence. Raw checkpoints remain outside Git and are never loaded by graded training.

References and saved evidence

[1] Jesse T. N. Roberts, instructor baseline and Project 1 harness: https://github.com/JesseTNRoberts/AI4200-imagenet32x32-baseline-model. Submission: https://github.com/TaylorTurnerIT/AI4200-imagenet32x32-baseline-model.
[2] Repository LOG.md; outputs/experiment-history/run-index.md; outputs/report-figures/accuracy-data.json and cifar10-training-comparison.json. Key runs: cifar10-20261023-134657-9294e112 (77), cifar10-20261024-151753-14698b08 (79), cifar10-20261026-215237-2ac3a7b3 (137), cifar10-20261026-221344-5d653ac6 (138), cifar10-20261026-224057-085c2c3b (139).
[3] outputs/final-evaluation/qualification.json and analysis/. Fresh qualification at b5ab6e1f75de4cfc257cf85b7e2535cfa217934c; runs mnist-c628ed51, cifar10-32ae8cde, imagenet32-8856b095. Scripts: scripts/analyze_submission.py and src/residual_mlp_baseline/analysis.py.
[4] outputs/window-stage-ablation/README.md and window-stage-ablation.json; scripts/ablate_window_stages.py. [5] LOG.md Change 146; outputs/pre-reduction-experiment/; imagenet32-prereduction-20261028-s0.
[6] MLP-Mixer: An all-MLP Architecture for Vision. https://arxiv.org/abs/2105.01601. [7] Activator: GLU Activation Function as the Core Component of a Vision Transformer. https://arxiv.org/abs/2405.15953.
[8] Gaussian Error Linear Units (GELUs). https://arxiv.org/abs/1606.08415.
[9] Decoupled Weight Decay Regularization. https://arxiv.org/abs/1711.05101. [10] On the Convergence of Adam and Beyond. https://arxiv.org/abs/1904.09237. Muon implementation reference: https://docs.pytorch.org/docs/stable/generated/torch.optim.Muon.html.